The Book vs. the `java` Command

Name: _____ Date: _____

This worksheet explores what happens when we run programs from a terminal with the `java` command.

In the book

```
record Time(int hour, int minute) {
  int toMinutes() {
    return hour * 60 + minute;
  }
  int minutesBetween(Time other) {
    return Math.abs(toMinutes() - other.toMinutes());
  }
}
int lunch = new Time(12, 0).minutesBetween(new Time(13, 30));
int overnight = new Time(22, 45).minutesBetween(new Time(1, 15));
```

Q1. Clicking Run in the reading shows each top-level name. Fill in what it shows.

`lunch` = _____ `overnight` = _____

Q2. `overnight` is meant to be the minutes from 10:45pm to 1:15am. Is the result what you expected? What would you have expected?

At a terminal

The same code, saved as `Time.java`, and run from the terminal (just like we ask in PA0):

```
$ java Time.java
Time.java:1: error: compact source file does not have main method in the form of void main() or
void main(String[] args)
record Time(int hour, int minute) {
^
1 error
error: compilation failed
```

Q3. What does `java` look for that this file doesn't have? Where did the book's display of `lunch` and `overnight` come from?

Q4. Here is a version with `main`. Fill in what the terminal prints.

```
record Time(int hour, int minute) { ... }  // as above

void main() {
  System.out.println(new Time(12, 0).minutesBetween(new Time(13, 30)));
  System.out.println(new Time(22, 45).minutesBetween(new Time(1, 15)));
}
```

```
$ java Time.java
```

_____

_____

`main` as a home for tests

```
import static org.junit.jupiter.api.Assertions.*;

record Time(int hour, int minute) { ... }  // as above

void main() {
  assertEquals(90, new Time(12, 0).minutesBetween(new Time(13, 30)));
  assertEquals(90, new Time(13, 30).minutesBetween(new Time(12, 0)));
}
```

```
$ java -cp junit-platform-console-standalone-1.13.4.jar Time.java
$
```

Q5. The run printed nothing. Did the tests pass? How can you tell? Why does this file need the `import` line and the `-cp` part of the command, when the book didn't?

Now `main` has three tests:

```
void main() {                                                    // line 12
  assertEquals(90, new Time(12, 0).minutesBetween(new Time(13, 30)));   // line 13
  assertEquals(150, new Time(22, 45).minutesBetween(new Time(1, 15)));  // line 14
  assertEquals(0, new Time(9, 0).minutesBetween(new Time(9, 0)));       // line 15
}
```

```
$ java -cp junit-platform-console-standalone-1.13.4.jar Time.java
Exception in thread "main" org.opentest4j.AssertionFailedError: expected: <150> but was: <1290>
  at org.junit.jupiter.api.AssertionFailureBuilder.build(AssertionFailureBuilder.java:151)
  ...
  at Time.main(Time.java:14)
```

Q6. Which test failed, and how do you know? Did the test on line 15 run?

Q7. Is the bug in the test or in `minutesBetween`? Could it be either one? Say what you would change in each case.

Q8. Write two more tests for `minutesBetween` in `main`. Work out the expected values by hand, and pick cases that are different from the ones above.

Q9. Could the `assertEquals` lines go at the top level, outside of `main`, the way `int lunch = ...` did in the book? What do you think `java` says?